

Guía práctica de Programación Orientada a Objetos

Python | Clases, objetos, constructores, encapsulación, herencia, herencia múltiple y MRO

Nombre: _____

Fecha: _____

Horario: 8:00 a. m. a 3:00 p. m.

Duración: 7 horas

Indicaciones

Resuelva los ejercicios en orden. Cree varios objetos para probar cada clase y utilice nombres claros para atributos y métodos. En los ejercicios de encapsulación, los datos privados no deben modificarse directamente desde fuera de la clase.

Bloque 1. Clases, objetos y métodos

1. Registro de teléfonos

Cree una clase Telefono que represente los equipos disponibles en una tienda.

Requisitos: atributos: marca, modelo y precio; método mostrar_informacion(); crear tres objetos.

2. Catálogo de libros

Cree una clase Libro para almacenar la información básica de un libro.

Requisitos: atributos: titulo, autor y paginas; métodos mostrar() y leer(); crear dos libros.

3. Datos de empleados

Diseñe una clase Empleado y muestre los datos de cada empleado mediante un método.

Requisitos: atributos: nombre, cargo y salario; método mostrar_datos(); crear tres empleados.

4. Estado de una computadora

Cree una clase Computadora que pueda encenderse y apagarse.

Requisitos: atributos: marca, ram y procesador; métodos encender() y apagar().

Bloque 2. Constructores

5. Información de vehículos

Implemente la clase Vehiculo utilizando el constructor `__init__()`.

Requisitos: atributos: marca, modelo y anio; método mostrar(); crear dos vehículos.

6. Registro de estudiantes

Cree una clase Estudiante cuya información se inicialice desde el constructor.

Requisitos: atributos: nombre, carrera y promedio; método mostrar_informacion().

7. Control de productos

Desarrolle una clase Producto que permita registrar un producto y reducir su existencia.

Requisitos: atributos: nombre, precio y stock; métodos mostrar() y vender(cantidad).

8. Mascotas de una veterinaria

Cree una clase Mascota con constructor y un método que muestre una presentación breve.

Requisitos: atributos: nombre, especie y edad; método presentarse().

Bloque 3. Encapsulación

9. Cuenta bancaria: consulta

Cree una clase CuentaBancaria. El saldo debe ser privado para evitar modificaciones directas.

Requisitos: atributos: titular y __saldo; método consultar_saldo().

10. Cuenta bancaria: depósitos

Agregue a la clase anterior un método para depositar dinero.

Requisitos: método depositar(monto); aceptar únicamente montos mayores que cero; mostrar el saldo actualizado.

11. Cuenta bancaria: retiros

Agregue un método que permita retirar dinero de forma controlada.

Requisitos: método retirar(monto); no aceptar montos menores o iguales a cero; no retirar más del saldo disponible.

12. Control de calificaciones

Cree una clase Alumno cuya nota se mantenga protegida.

Requisitos: atributos: nombre y __nota; métodos consultar_nota() y cambiar_nota(nueva_nota); aceptar notas entre 0 y 100.

Bloque 4. Herencia

13. Persona y profesor

Cree una clase padre Persona y una clase hija Profesor.

Requisitos: Persona: nombre y edad; Profesor: materia; usar super().__init__(); métodos presentarse() y enseñar().

14. Vehículo y carro

Cree una clase Carro que herede de Vehiculo.

Requisitos: Vehiculo: marca y modelo; Carro: numero_puertas; métodos mostrar() y encender().

15. Animales y sobrescritura

Cree una clase padre Animal con el método hablar(). Luego cree las clases Perro y Gato.

Requisitos: sobrescribir hablar() en cada clase hija; Perro debe mostrar Guau; Gato debe mostrar Miau.

16. Empleados especializados

Cree una clase padre Empleado y dos clases hijas: Programador y Contador.

Requisitos: Empleado: nombre y salario; Programador: lenguaje; Contador: departamento; usar super().__init__().

Bloque 5. Herencia múltiple

17. Pato nadador y volador

Cree las clases Nadador y Volador. Luego cree Pato heredando de ambas.

Requisitos: Nadador: método nadar(); Volador: método volar(); Pato debe usar ambos métodos.

18. Usuario con varias capacidades

Cree las clases Autenticacion y Permisos. Después cree UsuarioAdministrador heredando de ambas.

Requisitos: Autenticacion: iniciar_sesion(); Permisos: mostrar_permisos(); probar los dos métodos.

Bloque 6. Orden de resolución de métodos (MRO)

19. Orden de las clases padre

Cree dos clases, A y B, con un método saludar() diferente en cada una. Después pruebe primero class C(A, B) y luego class C(B, A).

Requisitos: ejecutar C().saludar(); mostrar C.mro(); explicar por qué cambia el resultado.

20. Herencia en forma de diamante

Cree las clases Persona, Empleado(Persona), Programador(Persona) y Senior(Empleado, Programador). Todas deben tener un método accion() con mensajes distintos.

Requisitos: crear un objeto Senior; ejecutar accion(); mostrar Senior.mro(); explicar el recorrido.

Actividad integradora

Sistema de biblioteca. Cree una clase padre Material con titulo y codigo. Luego cree las clases Libro, Revista y Periodico. Cada clase hija debe agregar un atributo propio y sobrescribir un método mostrar_detalle(). Finalmente, almacene varios objetos en una lista y recórrala para mostrar toda la información.